IITM Explainable Slotbook: Design Document

CS5013 course project • Nishanth S (EE23B049) and Sparsh Gupta (EE23B117)

Written up in November 2026 from our October design notes, the prompt log (P001–P017) and the commit history. It records the plan we started from, so it is in the future tense, and §4 shows how that plan changed.

Every semester our department gives each course a lecture slot on the institute timetable grid, mostly by hand-editing last year's slotbook. Nobody can say afterwards why a course got its slot, or what a clash would cost to fix. We will build a tool that produces the slotbook from the rules and preferences directly, never breaks a hard rule, keeps a documented penalty low, and explains every assignment. Alongside it, a small web portal will collect the inputs. We are deliberately leaving some things out: no language model decides or explains anything, rooms are not allocated, the demo portal has no login, and all data in the repository is synthetic.

1 Architecture

The system has two halves that meet only at plain CSV files (Figure 1). The **engine** is a Java 17 program, run from a command line or a small JavaFX viewer. The **portal** is a set of web pages with serverless functions on Vercel. Keeping the boundary at files means either half can be used, tested or replaced on its own, and a timetabler can always open the files in a spreadsheet.

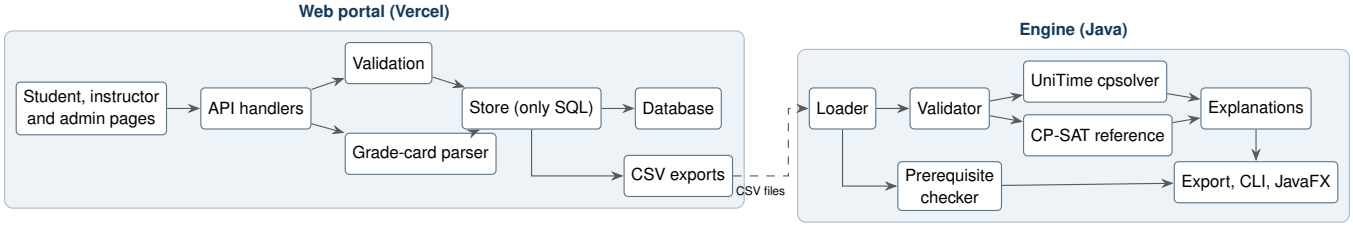


Figure 1: Architecture. Data flows left to right; the dashed arrow is the only link between the two halves.

Engine. Data flows one way: load, validate, solve, explain, export. Validation runs before any solving, and reports every problem with a specific code (e.g. `FIXED_CONFLICT`). Invalid input is never solved. Clashes are computed from real meeting times on the official IITM grid, since slots with different letters can overlap. The hard rules are:

- one slot per course, taken from its allowed set (or its fixed slot);
- no lecture over the course's own lab;
- no overlap between protected pairs of courses;
- no instructor teaching two overlapping meetings;
- no two lectures in the same room at once.

One objective, shared. A single `Objective` class defines the penalty, and every solver and the explanation code use it. A course costs the instructor's penalty for its slot (0–100, and 1 if they gave none), plus 2 if it moves away from last year's slot. Each overlapping pair costs 3 per overlapping meeting, times the pair's weight. The weight sums $\max(1,\ 4 - \text{worse rank})$ over the students taking both courses, so a clash between top choices hurts most.

Solvers behind an interface. We will start with an exact CP-SAT model (OR-Tools), which can prove optimality on small inputs. We then plan to wrap UniTime's *cpsolver* library, a local-search engine proven on real university timetables, and make it the default for scale. CP-SAT stays as the yardstick, so the heuristic's gap from the optimum is measured, not assumed.

Explanations from the model, not the solver. After solving, we recompute everything from the problem and the final assignment. For the chosen slot we give the rules it satisfies and the penalties it pays. For every other slot we give the rule it breaks, or its estimated extra cost. Because the explanation code never looks inside the solver, explanations are identical whichever solver ran.

Portal. Requests pass through pure validation code into a store module, the only code that touches the database; every query is parameterised. Storage starts as a Google Sheet for the demo, and moves to Postgres once we need relational rules and concurrent writers. There, the validation rules are repeated as foreign keys and `CHECK` constraints, and each replacing write is one transaction. Prerequisites are advisory: a student's uploaded grade card is parsed (grades U, W and I don't count as completed), missing prerequisites produce warnings, and nothing is blocked or sent to the solver.

2 Module split

Each module has one job, and dependencies point one way: `model` depends on nothing, and everything else depends on `model`. The CLI and the viewer share `service`, so neither contains solver logic.

Module	Main classes	Responsibility
Engine (`in.ac.iitm.slotbook`)		
`model`	`Course`, `Slot`, `TimeGrid`, `TimetableProblem`, `TimetableSolution`	Immutable records; real meeting times; student pair weights
`io`	`DatasetLoader`, `CsvSupport`, `SlotbookExporter`, `PrerequisiteLoader`	Read the five CSVs (plus optional prerequisite files) and write the exports
`validation`	`InputValidator`, `PrerequisiteChecker`	Reject invalid input with coded issues; compute advisory prerequisite warnings
`solver`	`TimetableSolver`, `Objective`, `OrToolsTimetableSolver`, `unitime/*`	The solver interface, the shared objective, CP-SAT, and the cpsolver mapping and search
`explain`	`ExplanationBuilder`	Per-course reasons and classified alternatives
`service`, `cli`, `ui`	`SlotbookService`, `SlotbookCli`, `SlotbookApplication`	One facade; command line and JavaFX on top of it
`testing` (tests)	`ConstraintChecker`, `PenaltyCalculator`, `BruteForceSolver`, `ScenarioGenerator`	Independent oracles and generated scenarios, kept out of production code
Portal (`portal/`)		
`api/*.js`	`submit`, `admin`, `results`, `solver`, `health`	HTTP handlers: parse, call domain and store, map errors to status codes
domain	`_validation`, `_gradecard`, `_prerequisites`, `_completions`	Pure rules; the grade-card parser; the completion-source registry
storage	`_store`, `_db`, `db/migrations`	All SQL; connection pool; numbered schema changes
`public/`	Student, professor, admin and results pages	Forms and dashboards; no business rules in the browser

3 Test plan

We test against oracles written separately from the code under test, rather than asking the code whether it is right.

Level	What we test	Checked against
Unit	CSV parsing, every validation code, the objective, explanations, grade-card rows, the prerequisite rule	Hand-computed expected values
Solver correctness	Both solvers on fixed fixtures and randomly generated scenarios	Independent constraint checker and penalty calculator
Optimality	CP-SAT on small seeded problems	Exhaustive brute-force search
Cross-solver	cpsolver against CP-SAT on every scenario	CP-SAT's proven optimum; gap recorded
Contract	Portal CSV exports; portal against CLI prerequisite warnings	Engine's loader headers; one shared golden file
Integration	Store on a real Postgres: rollback, concurrency, constraints	Database state afterwards
Stress	Seeded random input, corrupted PDFs, large volumes, hundreds of simultaneous requests	Invariants (never a 500 or a half-written submission); naive re-implementations
Manual	JavaFX viewer; portal pages in a browser, console errors included	A checklist

Four rules apply throughout:

- Never assert one exact timetable when several are equally good.
- Seed every random test, so a failure can be replayed.
- Show that every bug-fix test fails on the old code.
- List every test, with what it checks, in the test plan.

GitHub Actions runs the Java suite, the portal suites (with a Postgres service), CLI runs with both solvers, and a man-page check on every push and pull request.

4 Revised milestone plan

Our first plan (prompts P001–P005) ran straight to the mid-evaluation: architecture and synthetic data, the CP-SAT engine with explanations, tests, then evaluator documentation. We revised it as we learned. Comparing our model with UniTime showed that its search scales much better, and that our first engine missed a room clash and used approximate slot times, so we added a cpsolver milestone with real IITM meeting times. Typing preferences up by hand was clearly going to be the bottleneck, so we added the portal. Later, concurrent writers and prerequisites made relational storage necessary, so Postgres replaced the Sheet. Finally, we added a hardening milestone before deployment.

	Milestone	**Done when**	**Change from first plan**
M1	Engine v1: CP-SAT, explanations, CLI, JavaFX, CI	Feasible fixture solves; infeasible one reports `FIXED_CONFLICT`; hard rules hold on every output	As planned
M2	Testing framework	CP-SAT equals brute force on every seed; oracles agree with both solvers	Expanded (oracles, generated scenarios)
M3	Engine v2: cpsolver default, real IITM times, rooms, labs	cpsolver's score equals our objective; gap from CP-SAT measured on all scenarios	**New** (after the UniTime comparison)
M4	Demo portal on Google Sheets	Exports match the engine's CSVs byte for byte	**New**
M5	Mid-evaluation	Showcase dataset, three-page practices PDF, live code change rehearsed	As planned
M6	v3: Postgres, grade-card prerequisite warnings, man page	Portal and CLI warnings equal the golden file; integration tests pass on Postgres	**New** (replaces the Sheet)
M7	Hardening and deployment	No server errors under stress; deployed with a configured database	**New**

Main risks. The heuristic may fall short of the optimum; CP-SAT stays as the yardstick. Our objective could drift from cpsolver's score, so a test checks they are equal. Simultaneous submissions could corrupt data; transactions, constraints and stress tests guard against that. Real student data must never reach the repository: test grade cards are generated and marked, and a check rejects any that aren't. Until there is institute login and a consent policy, the portal collects no real data.